



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 01 – Lecture 01: Introduction to ML & the Python ML Stack

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Topic focus: **What is machine learning?** what kinds there are, and the tools we will use

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.



Quick Links

Course

What is ML

Types

Python Stack

Summary

Agenda

Course Overview

What is Machine Learning?

Types of Machine Learning

The Python ML Stack

Summary

Learning Outcomes

By the end of this lecture, you should be able to:

- Explain what **machine learning** is and when it is the right tool.

Mapped to **CO1** (core concepts) and **CO2** (build models with standard libraries).

Learning Outcomes

By the end of this lecture, you should be able to:

- Explain what **machine learning** is and when it is the right tool.
- State Mitchell's definition in terms of **task, performance measure and experience**.

Mapped to **CO1** (core concepts) and **CO2** (build models with standard libraries).

Learning Outcomes

By the end of this lecture, you should be able to:

- Explain what **machine learning** is and when it is the right tool.
- State Mitchell's definition in terms of **task, performance measure and experience**.
- Distinguish **supervised, unsupervised** and **reinforcement** learning, and name the task type for a given problem.

Mapped to **CO1** (core concepts) and **CO2** (build models with standard libraries).

Learning Outcomes

By the end of this lecture, you should be able to:

- Explain what **machine learning** is and when it is the right tool.
- State Mitchell's definition in terms of **task, performance measure and experience**.
- Distinguish **supervised, unsupervised** and **reinforcement** learning, and name the task type for a given problem.
- Describe the roles of **NumPy, Pandas** and **scikit-learn**, and run a complete model in a dozen lines.

Mapped to **CO1** (core concepts) and **CO2** (build models with standard libraries).

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, and the Python stack.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, and the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, and the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adadelata, Adam.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, and the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adadelata, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation, imbalance.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, and the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adadelata, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation, imbalance.
- **Unit V:** Regression — linear, logistic, multiple, regularization.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, and the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adadelata, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation, imbalance.
- **Unit V:** Regression — linear, logistic, multiple, regularization.
- **Unit VI:** Classification — KNN, trees, Bayes, ensembles, neural networks, SVM, evaluation.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, and the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adadelata, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation, imbalance.
- **Unit V:** Regression — linear, logistic, multiple, regularization.
- **Unit VI:** Classification — KNN, trees, Bayes, ensembles, neural networks, SVM, evaluation.
- **Unit VII:** Clustering — hierarchical, K-means, K-medoids, CLARA, DBSCAN.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, and the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adadelata, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation, imbalance.
- **Unit V:** Regression — linear, logistic, multiple, regularization.
- **Unit VI:** Classification — KNN, trees, Bayes, ensembles, neural networks, SVM, evaluation.
- **Unit VII:** Clustering — hierarchical, K-means, K-medoids, CLARA, DBSCAN.

Units II and III come early on purpose: once you know **what we minimise** and **how**, Units V–VII are variations on one theme.

Course Structure and Assessment

Course code	CSAI2017P
L-T-P-C	4-0-1-5
Units	7
Theory	24 lectures / 40 sessions
Lab	15 experiments

Component	Marks
Theory assignments (2, each with a test)	5
Theory quizzes (10 short tests)	10
Project — individual quiz	10
Project — group submission state	5
Lab assignments (10)	10
Lab quizzes (10)	10
Mid-Semester examination	20
End-Semester examination	30
Total	100

Read this carefully

The project is worth 15, and **10 of those 15 are individual**. A strong contributor in a weak group can still score well; a passenger cannot.

Indicative. The final component split is subject to departmental approval and will be confirmed on the LMS.

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced** — expect it within a week or two.

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced** — expect it within a week or two.
- It **carries no marks** and is not part of any component.

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced** — expect it within a week or two.
- It **carries no marks** and is not part of any component.
- Its purpose: to let the teaching team see **how you approach a problem**, so the short tests through the semester can be framed to suit how you work.

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced** — expect it within a week or two.
- It **carries no marks** and is not part of any component.
- Its purpose: to let the teaching team see **how you approach a problem**, so the short tests through the semester can be framed to suit how you work.
- It is **not** a ranking and **not** an ability test. Nothing to revise.

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced** — expect it within a week or two.
- It **carries no marks** and is not part of any component.
- Its purpose: to let the teaching team see **how you approach a problem**, so the short tests through the semester can be framed to suit how you work.
- It is **not** a ranking and **not** an ability test. Nothing to revise.

How to answer it

Show your working. An incomplete answer that makes its reasoning visible is **more** useful to us than a bare correct one. If you do not know something, write what you would need to find out — that is a useful answer, not a blank one.

Abbreviations (Used in This Lecture)

- **ML:** Machine Learning

Abbreviations (Used in This Lecture)

- **ML:** Machine Learning
- **AI:** Artificial Intelligence

Abbreviations (Used in This Lecture)

- **ML:** Machine Learning
- **AI:** Artificial Intelligence
- **DL:** Deep Learning

Abbreviations (Used in This Lecture)

- **ML:** Machine Learning
- **AI:** Artificial Intelligence
- **DL:** Deep Learning
- **EDA:** Exploratory Data Analysis (lab focus)

Abbreviations (Used in This Lecture)

- **ML:** Machine Learning
- **AI:** Artificial Intelligence
- **DL:** Deep Learning
- **EDA:** Exploratory Data Analysis (lab focus)
- **MSE:** Mean Squared Error (Unit II)

Abbreviations (Used in This Lecture)

- **ML:** Machine Learning
- **AI:** Artificial Intelligence
- **DL:** Deep Learning
- **EDA:** Exploratory Data Analysis (lab focus)
- **MSE:** Mean Squared Error (Unit II)
- **KNN:** K-Nearest Neighbours (Unit VI)

The Shape of the Problem

- **Programs we can write by hand:** sort a list, parse a date, compute a payroll. We know the rule; we encode the rule.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

The Shape of the Problem

- **Programs we can write by hand:** sort a list, parse a date, compute a payroll. We know the rule; we encode the rule.
- **Programs we cannot write by hand:** decide whether an email is spam, read a handwritten digit, tell whether a scan shows a tumour.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

The Shape of the Problem

- **Programs we can write by hand:** sort a list, parse a date, compute a payroll. We know the rule; we encode the rule.
- **Programs we cannot write by hand:** decide whether an email is spam, read a handwritten digit, tell whether a scan shows a tumour.
- In the second group we **recognise** the right answer but cannot state the rule that produces it.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

The Shape of the Problem

- **Programs we can write by hand:** sort a list, parse a date, compute a payroll. We know the rule; we encode the rule.
- **Programs we cannot write by hand:** decide whether an email is spam, read a handwritten digit, tell whether a scan shows a tumour.
- In the second group we **recognise** the right answer but cannot state the rule that produces it.

Working idea for this course

Machine learning is what you reach for when you have **examples of the answer** but not a **statement of the rule**. The examples are used to find the rule.

That is the *supervised* case, which is most of this course. The other two families relax it — we get to them shortly.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Two Ways to Build a Spam Filter

Rule-based

```
def classify(subject, sender, bangs):  
    if "free money" in subject:  
        return "SPAM"  
    if sender in blocklist:  
        return "SPAM"  
    if bangs > 5:  
        return "SPAM"  
    return "HAM"
```

Spammer writes "fr33 m0ney". You add a rule.
They adapt. You add a rule.

Discussion

What changed is the **maintenance strategy**, not only the code. But the rule-based filter keeps one property the learned one loses — can you name it?

Learned

```
model.fit(emails, labels)  
model.predict(new_email)
```

Spammer adapts. You collect newly labelled mail.
You refit.

A Working Definition (Mitchell, 1997)

Definition

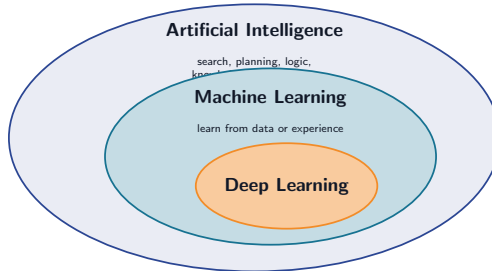
A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

	Task T	Experience E	Performance P
Spam filter	classify email	labelled inbox	fraction correctly classified
House prices	predict price	past sales records	mean squared error
Game agent	choose a move	games played	win rate

Practical takeaway

The definition forces you to name P **before** you start. Many failed ML projects failed here, not in the modelling.

Where ML Sits: $AI \supset ML \supset DL$



Careful

Deep learning is a **subset** of ML, not a synonym and not an automatic upgrade. On tabular data of the size you meet in this course, a gradient-boosted tree usually beats a neural network — and trains in seconds.

Why ML “Works” Today (Three Drivers)

- **Data:** digitised transactions, sensors, images and logs — the examples exist and are cheap to store.

Why ML “Works” Today (Three Drivers)

- **Data:** digitised transactions, sensors, images and logs — the examples exist and are cheap to store.
- **Compute:** parallel hardware made the arithmetic of Unit III affordable at scale.

Why ML “Works” Today (Three Drivers)

- **Data:** digitised transactions, sensors, images and logs — the examples exist and are cheap to store.
- **Compute:** parallel hardware made the arithmetic of Unit III affordable at scale.
- **Algorithms and tooling:** better optimisers and regularisation, plus libraries that put all of it three lines away.

Why ML “Works” Today (Three Drivers)

- **Data:** digitised transactions, sensors, images and logs — the examples exist and are cheap to store.
- **Compute:** parallel hardware made the arithmetic of Unit III affordable at scale.
- **Algorithms and tooling:** better optimisers and regularisation, plus libraries that put all of it three lines away.

Most core ideas here are decades old — least squares dates from about 1805. What changed is that data and compute caught up with the mathematics.

Applications — and the Unit that Covers Them

Domain	Task	Unit
Finance	credit scoring, fraud detection	VI
Healthcare	diagnosis from imaging, signal classification	VI, VII
Retail	churn prediction, customer segmentation	V, VI, VII
Agriculture	crop disease from leaf images	VI
Web	spam filtering, recommendation	VI, VII
Operations	demand and price forecasting	V

You will build this column

Most rows above appear as a lab experiment. By the end of the lab programme you will have written them.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a lookup, not a prediction.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a lookup, not a prediction.
- **No labelled examples** exist and none can realistically be obtained.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a lookup, not a prediction.
- **No labelled examples** exist and none can realistically be obtained.
- **Errors are unacceptable** and you cannot bound them.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a lookup, not a prediction.
- **No labelled examples** exist and none can realistically be obtained.
- **Errors are unacceptable** and you cannot bound them.
- **Every decision must be explained** to a regulator, and a simple rule would do nearly as well.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a lookup, not a prediction.
- **No labelled examples** exist and none can realistically be obtained.
- **Errors are unacceptable** and you cannot bound them.
- **Every decision must be explained** to a regulator, and a simple rule would do nearly as well.
- **Training data will not resemble deployment.** A model trained on last year's customers predicts last year's customers.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a lookup, not a prediction.
- **No labelled examples** exist and none can realistically be obtained.
- **Errors are unacceptable** and you cannot bound them.
- **Every decision must be explained** to a regulator, and a simple rule would do nearly as well.
- **Training data will not resemble deployment.** A model trained on last year's customers predicts last year's customers.

Discussion

A hospital wants to predict which patients will miss an appointment. Which of the five is most likely to bite, and why?

The Three Families

Supervised

learn a mapping

$$x \mapsto y$$

labelled data $\{(x_i, y_i)\}$

Unsupervised

find structure
in x alone

unlabelled data $\{x_i\}$

Reinforcement

learn a policy
from reward

environment + reward

The question that decides which family you are in

Do you already have the answers? Yes, for past examples $\rightarrow$ supervised. No, and you want structure $\rightarrow$ unsupervised. No, but you can act and be scored $\rightarrow$ reinforcement.

Supervised Learning

Given $\{(x_i, y_i)\}_{i=1}^n$, find f such that $f(x) \approx y$ on data **not seen during training**.

- **Regression** — y is continuous: house price, stock value, temperature, time to failure.
(Unit V; lab experiments 3–4)

Supervised Learning

Given $\{(x_i, y_i)\}_{i=1}^n$, find f such that $f(x) \approx y$ on data **not seen during training**.

- **Regression** — y is continuous: house price, stock value, temperature, time to failure. (Unit V; lab experiments 3–4)
- **Classification** — y is a category: spam or not, digit 0–9, benign or malignant. (Unit VI; lab experiments 5–9, 11–12)

Supervised Learning

Given $\{(x_i, y_i)\}_{i=1}^n$, find f such that $f(x) \approx y$ on data **not seen during training**.

- **Regression** — y is continuous: house price, stock value, temperature, time to failure. (Unit V; lab experiments 3–4)
- **Classification** — y is a category: spam or not, digit 0–9, benign or malignant. (Unit VI; lab experiments 5–9, 11–12)

The whole difficulty in one sentence

Fitting the training data is easy — a model can simply memorise it. The task is to be right on data never seen before. That single requirement is why Unit IV (splitting, cross-validation) and the over-fitting material in Unit V exist.

Unsupervised and Reinforcement Learning

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; lab experiment 13)

Unsupervised and Reinforcement Learning

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; lab experiment 13)
- **Dimensionality reduction** — fewer features carrying much the same information. (Unit IV)

Unsupervised and Reinforcement Learning

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; lab experiment 13)
- **Dimensionality reduction** — fewer features carrying much the same information. (Unit IV)
- **Anomaly detection** — identify what does not fit. (lab experiment 10)

Unsupervised and Reinforcement Learning

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; lab experiment 13)
- **Dimensionality reduction** — fewer features carrying much the same information. (Unit IV)
- **Anomaly detection** — identify what does not fit. (lab experiment 10)
- **Reinforcement learning** — an agent acts in an environment and receives reward; no labelled answers, only consequences, often delayed. Robotics, game playing, control.

Unsupervised and Reinforcement Learning

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; lab experiment 13)
- **Dimensionality reduction** — fewer features carrying much the same information. (Unit IV)
- **Anomaly detection** — identify what does not fit. (lab experiment 10)
- **Reinforcement learning** — an agent acts in an environment and receives reward; no labelled answers, only consequences, often delayed. Robotics, game playing, control.

Why unsupervised is harder to judge

Unsupervised results have no ground truth to check against, so evaluating a clustering is genuinely harder than evaluating a classifier. Unit VII therefore spends real time on cluster validity.

Naming the Problem (Worked Exercise)

Situation

Family

Task

10 000 past loans marked repaid or defaulted;
score a new applicant
5 years of daily sales; forecast next month
50 000 customer records, no labels; find natural
groups
Card transactions; flag the unusual ones
A warehouse robot that must learn a route

Naming the Problem (Worked Exercise)

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant 5 years of daily sales; forecast next month 50 000 customer records, no labels; find natural groups Card transactions; flag the unusual ones A warehouse robot that must learn a route	supervised	classification

Naming the Problem (Worked Exercise)

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant	supervised	classification
5 years of daily sales; forecast next month	supervised	regression
50 000 customer records, no labels; find natural groups		
Card transactions; flag the unusual ones		
A warehouse robot that must learn a route		

Naming the Problem (Worked Exercise)

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant	supervised	classification
5 years of daily sales; forecast next month	supervised	regression
50 000 customer records, no labels; find natural groups	unsupervised	clustering
Card transactions; flag the unusual ones		
A warehouse robot that must learn a route		

Naming the Problem (Worked Exercise)

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant	supervised	classification
5 years of daily sales; forecast next month	supervised	regression
50 000 customer records, no labels; find natural groups	unsupervised	clustering
Card transactions; flag the unusual ones	unsupervised	anomaly detection
A warehouse robot that must learn a route		

Naming the Problem (Worked Exercise)

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant	supervised	classification
5 years of daily sales; forecast next month	supervised	regression
50 000 customer records, no labels; find natural groups	unsupervised	clustering
Card transactions; flag the unusual ones	unsupervised	anomaly detection
A warehouse robot that must learn a route	reinforcement	policy learning

Discussion

Row 4 could also be posed as supervised. What would you need for that, and why might a bank not have it?

Three Libraries, Three Jobs

scikit-learn — models, preprocessing, evaluation, pipelines

Pandas — tabular data: load, clean, reshape, group

NumPy — the ndarray and fast array arithmetic

Everything above rests on the layer below

A Pandas DataFrame holds NumPy arrays; a scikit-learn estimator consumes NumPy arrays. Understanding the bottom layer is what lets you debug the top one.

Matplotlib and Seaborn join the stack for plotting from Unit IV onwards.

NumPy — Vectorisation is the Point

Python loop

```
total = 0.0
for i in range(len(y)):
    d = y[i] - yhat[i]
    total += d * d
mse = total / len(y)
```

NumPy

```
import numpy as np
mse = np.mean((y - yhat) ** 2)
```

Shorter, and typically 10–100× faster: the loop runs in compiled code over a contiguous block of memory.

Note what you just wrote

That is the **Mean Squared Error** — the first loss function of Unit II. Every optimiser in Unit III exists to make it smaller.

Essentials for lab 1: `np.array`, `.shape`, `.reshape`, broadcasting, `axis`, boolean masks.

Pandas — the DataFrame

```
import pandas as pd

df = pd.read_csv("data.csv")
df.head() # look at it first, always
df.info() # dtypes and non-null counts
df.describe() # numeric summary
df.isna().sum() # where is the data missing?

df["price_per_sqft"] = df["price"] / df["area"]
df.groupby("city")["price"].mean()
```

Habit to build now

Run head, info, describe and isna().sum() before you model anything. Unit IV is the disciplined version of that reflex.

scikit-learn — One API for Every Model

```
est.fit(X, y) # learn  
est.predict(X) # apply  
est.score(X, y) # evaluate
```

```
tr.fit(X) # transformers  
tr.transform(X)
```

Why this matters

Swapping a decision tree for an SVM changes **one line**. Learn the API once here and every model in Units V–VII is already familiar.

The rule you must not break

Call `fit` on training data only — never on the test set, not even for scaling. Doing so leaks the answer and inflates your score. This is the most common bug in student ML code.

End to End in Twelve Lines

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

Read what each argument protects you from

`stratify=y` preserves the class balance. `random_state` makes it reproducible. `make_pipeline` fits the scaler *inside* the training portion only — the leak-proof way.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, companion notebooks.; McKinney, *Python for Data Analysis* (3e), O'Reilly, 2022.; Scikit-learn user guide, accessed 2026-08-04.; Matplotlib and Seaborn documentation, accessed 2026-08-04.

Before the First Lab

Set up your environment

1. Install Python 3.11+ (Anaconda or venv).
2. `pip install numpy pandas scikit-learn matplotlib seaborn jupyter`
3. Launch Jupyter; run the twelve-line Iris script.
4. Print the version of each library.

Lab assignment LA1

Experiments 1–2: environment setup, then preprocessing and EDA. Due after lab session 2; quiz LQ1 in lab session 3.

Lab quizzes test *your* code

They ask you to read, trace, debug and modify what you submitted. Copied code is hard to defend in three minutes.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure before modelling.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure before modelling.
- **Supervised** (regression, classification), **unsupervised** (clustering, dimensionality reduction, anomaly detection), **reinforcement** (policy from reward).

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure before modelling.
- **Supervised** (regression, classification), **unsupervised** (clustering, dimensionality reduction, anomaly detection), **reinforcement** (policy from reward).
- The goal is never to fit the training data — it is to **generalise**.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure before modelling.
- **Supervised** (regression, classification), **unsupervised** (clustering, dimensionality reduction, anomaly detection), **reinforcement** (policy from reward).
- The goal is never to fit the training data — it is to **generalise**.
- **NumPy** → **Pandas** → **scikit-learn**, with one `fit/predict` API for every model.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure before modelling.
- **Supervised** (regression, classification), **unsupervised** (clustering, dimensionality reduction, anomaly detection), **reinforcement** (policy from reward).
- The goal is never to fit the training data — it is to **generalise**.
- **NumPy** → **Pandas** → **scikit-learn**, with one `fit`/`predict` API for every model.
- **Never fit on the test set.**

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Next Lecture and Reading

Next — L2: Loss functions (Unit II)

Putting a number on how wrong a prediction is: MSE, MAE and Huber for regression; cross-entropy for classification; hinge and triplet for margins. That number is what everything else in the course minimises.

- Müller & Guido, *Introduction to Machine Learning with Python*, Ch. 1.
- Bishop, *Pattern Recognition and Machine Learning*, Ch. 1 (§1.1).
- Murphy, *Probabilistic Machine Learning: An Introduction*, Ch. 1 (open access).
- James et al., *ISL with Python*, Ch. 1–2 (open access).
- Scikit-learn user guide, “Getting Started”.

Free copies: statlearning.com (ISLP) • probml.github.io/pml-book (Murphy) • Bishop PRML via Microsoft Research.



Thank You

